

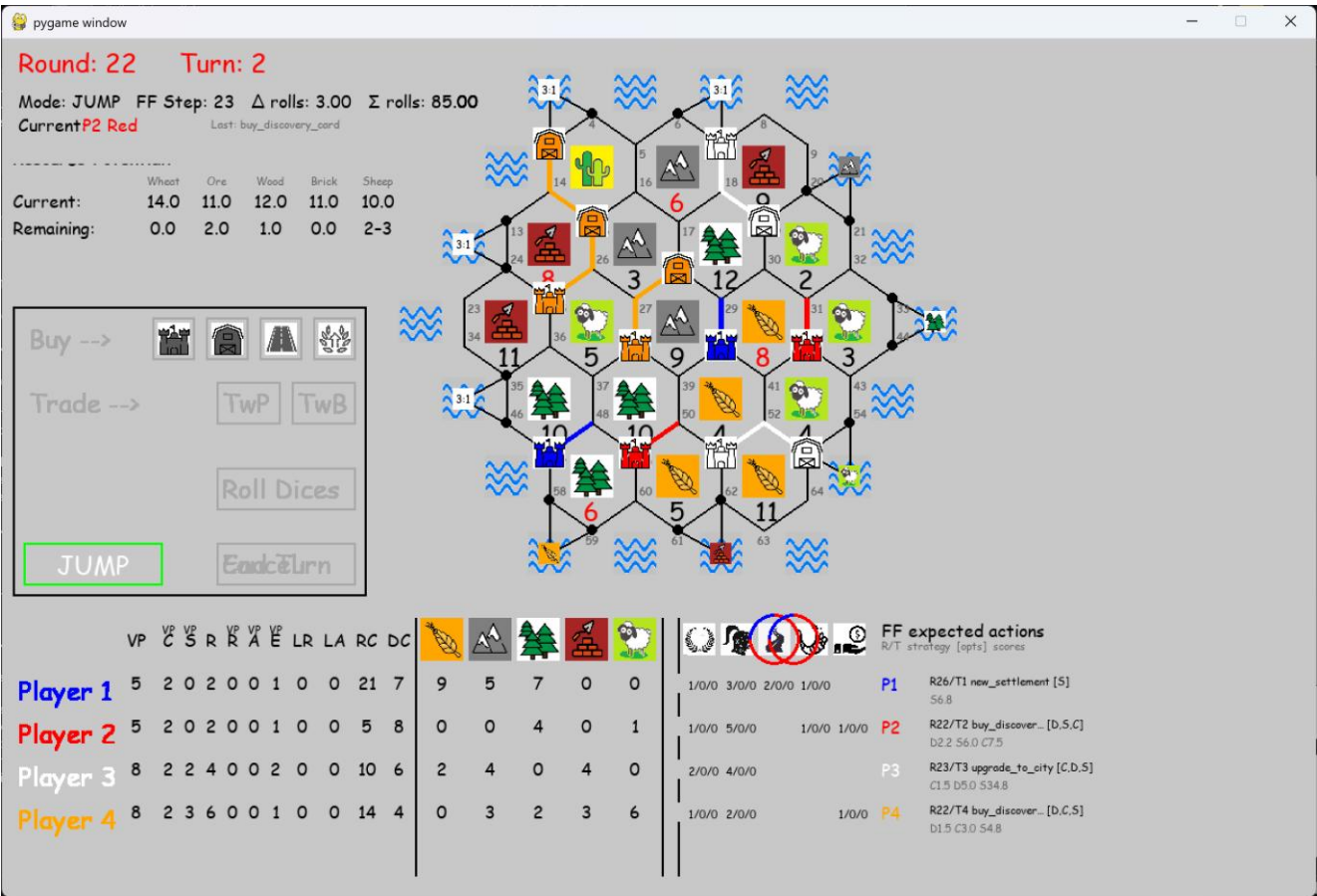
Catan Game Gen3 Experiment

Technical Overview v016 - Expected-Hand Driven Resource Timing

Anthony van Tilburg
May 2026

Version v016 milestone

Version v016 is best understood as the Expected-Hand version of the project. The repository still contains substantial Markov-related code, because internal v014 and v015 development moved in that direction before Expected-Hand became dominant. v014 and v015 are internal project milestones, not separately shared public GitHub releases. The public v016 repository carries forward those components, but the active timing direction is now dominated by Expected-Hand Feasibility Estimation. Human control is strongest during Initial Placement; after that, the user mainly advances the Expected-Hand engine with PLAY or JUMP rather than choosing every Execution-phase action manually.



Contents

1. Introduction
 2. Project status in v016
 3. Three-document GitHub guide
 4. System architecture
 5. GUI and interaction design
 6. Initial Placement phase
 7. Human interaction and automated Execution flow
 8. Expected-Hand Feasibility Estimation
 9. Markov status in v016
 10. Strategy and victory-path evaluation
 11. Fast-forward, JUMP, PLAY, and BEST NOW
 12. Trade, ports, and exact execution guards
 13. Debugging, reports, and explainability
 14. Known limitations and next steps
 15. Acknowledgements and external inspiration
- Annex A. Module overview
- Annex B. Key classes and responsibilities

1. Introduction

This overview describes an interim experiment within the Catan Game Gen3 project. This experiment is named v016 and is based on an earlier version (v010b), which was focused on constructing the board, rendering the scoreboard, enabling human Initial Placement, and adding stronger AI placement algorithms. Later versions (v032) moved into the Execution phase, where the engine rolls dice, trades, builds, buys development cards, and uses action recommendations. Version v016 keeps those foundations but shifts the technical center of gravity toward Expected-Hand Feasibility Estimation.

The game is implemented in Python with Pygame. It models a Catan-style board with 19 land tiles, surrounding sea tiles, intersections, roads, harbors, resource production, player hands, and a visible scoreboard. The user interface supports human-guided Initial Placement and automated continuation: after Initial Placement, the human player mainly uses PLAY and JUMP to advance the Expected-Hand-driven engine and observe how well the chosen starting positions perform.

Plain-English summary

Earlier versions mainly asked: "Where should a player place the first settlements?" v016 asks a broader question: "Given the current hand, board, production, ports, and strategy requirements, how many own turns until a useful action is feasible?"

2. Project status in v016

- Initial Placement remains supported with pips, ports, five-strategy logic, and Markov-style evaluation paths.
- After Initial Placement, Execution-phase control is intentionally limited: the user mainly presses PLAY or JUMP, while the engine handles rolls, production, robber flow, trades, builds, development-card buys, and turn advancement according to Expected-Hand and strategy recommendations.
- Expected-Hand timing has become the dominant resource-timing approach for estimating future affordability.
- Markov code remains in the repository but is mostly legacy, comparison, or fallback infrastructure in v016.
- Strategy and victory-path evaluation connects resource requirements, action candidates, expected turns, and continuation value.
- PLAY remains protected by exact legality and affordability checks. Expected-Hand predictions do not directly execute illegal actions.

The project should be presented as an educational AI/game-engine project. It is not a finished commercial Catan clone. Its value is in the progressive architecture: from static placement heuristics to Markov-inspired timing to a faster and more explainable Expected-Hand timing engine.

3. Three-document GitHub guide

For GitHub readers, v016 benefits from three different documents. They should not duplicate each other. The README should be short, this overview should explain the software architecture, and the Expected-Hand paper should explain the algorithmic research path.

Document	Purpose	Best audience
README.md	Quick landing page: what the project is, how to run it, and what the main features are.	First-time visitor
Technical Overview v016	Practical map of the current software: modules, GUI flow, Execution flow, Expected-Hand, and Markov status.	Reviewer, recruiter, future maintainer

Document	Purpose	Best audience
Paper leading to Expected-Hand Feasibility Estimation	Research and design explanation of the path from pips to Markov to Expected-Hand timing.	Reader interested in algorithms and reasoning

4. System architecture

The repository separates the board and game rules from rendering, interaction, timing, and strategy analysis. This split matters because board state changes should live in core game logic, GUI modules should draw and route input, and strategy modules should recommend actions rather than directly manipulating the screen.

Layer	Purpose	Representative modules
Core model	Represents board geometry, players, resources, roads, settlements, cities, ports, game state, and constants.	core/board.py, core/player.py, core/game.py, core/constants.py
Initial placement	Chooses first settlements and roads using pips, ports, five-strategy scoring, and Markov-style evaluation.	core/initial_placement.py, core/algorithms_initial_placement.py, core/markov_evaluator.py
Expected-Hand timing	Estimates when an action becomes payable from current hand, production pips, trade rates, and confidence settings.	core/resource_time_estimator.py
Strategy and victory-path evaluation	Compares candidate actions against resource requirements, Expected-Hand timing, and longer-term victory-path needs.	core/resource_time_estimator.py, core/victory_path_evaluator.py, core/action_evaluator.py, catan_142_ways_resource_requirements.csv
Execution orchestration	Runs selected actions, refreshes state, supports PLAY/JUMP behavior, and prevents stale recommendations.	core/fast_forward.py, core/action_evaluator.py, core/game.py
GUI and routing	Draws board, scoreboard, buttons, panels, highlights, animations, and routes mouse clicks.	gui/gui.py, gui/event_handler.py, gui/gui_human_player.py, gui/gui_constants.py

5. GUI and interaction design

The GUI keeps the style of earlier Gen3 versions: a central hexagonal board, a button panel, a scoreboard, and information/debug areas. The board shows tiles, number chits, ports, intersections, roads, settlements, cities, and the robber. During Initial Placement, the human player receives clickable choices. After Initial Placement, the button panel mainly lets the user advance automated Expected-Hand execution through PLAY or JUMP.

GUI area	Role in v016
Playboard	Shows the Catan-style board, legal targets, pieces, ports, robber, and visual highlights.

GUI area	Role in v016
Button panel	Activates phase-relevant controls. During Initial Placement it supports human choices; after Initial Placement, PLAY/JUMP are the main controls for advancing automated Expected-Hand execution.
Scoreboard	Shows victory points, pieces, resource cards, development cards, and player status.
Events panel	Shows what happened recently, such as rolls, resource income, robber events, and builds.
Execution / debug panel	Shows strategy and timing hints, including wait/action suggestions and BEST NOW-style recommendations.

This design makes the project visually inspectable. The goal is not only to let actions happen, but to make it clear why an action is available, why a target is highlighted, and why a recommendation was selected.

6. Initial Placement phase

Initial Placement remains an important part of the project. It is the point where a player creates the first production profile, which later drives Expected-Hand timing. A strong pair of starting settlements can reduce the expected turns to roads, settlements, cities, and development cards. A weak placement may force the engine to rely on trades or long waits.

Algorithm	What it values	Role in v016
Max Pips	Strong dice-production probability.	Simple baseline and sanity check.
Max Pips + Ports	Production plus access to useful harbors.	Adds basic trading awareness.
5 Weighted Strategic Strategies	Balanced production, Wood/Brick expansion, Wheat/Ore city path, development-card path, and monopoly-style pressure.	Preferred explainable strategy comparison for Initial Placement.
Markov-style evaluator	Longer-term expected value and uncertainty modeling.	Retained as advanced/historical infrastructure, not the dominant v016 timing engine.

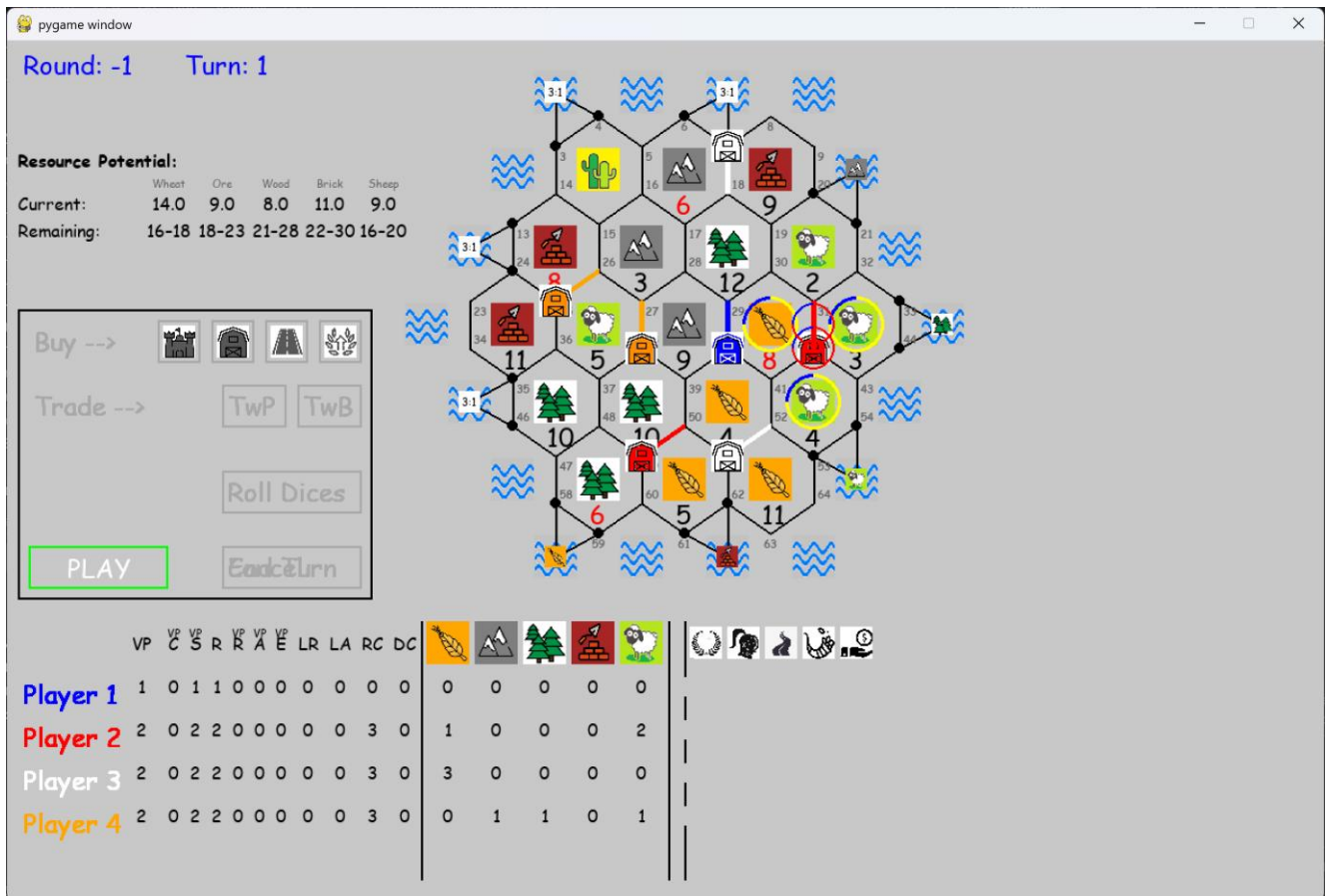


Figure 2. Red has just completed its second placement. It is now P1 Blue's turn to complete the Initial Placement phase.

Why Initial Placement still matters

Expected-Hand timing is only as good as the player profile it receives. The current hand, production pips, and port access all come from earlier placement and build decisions.

7. Human interaction and automated Execution flow

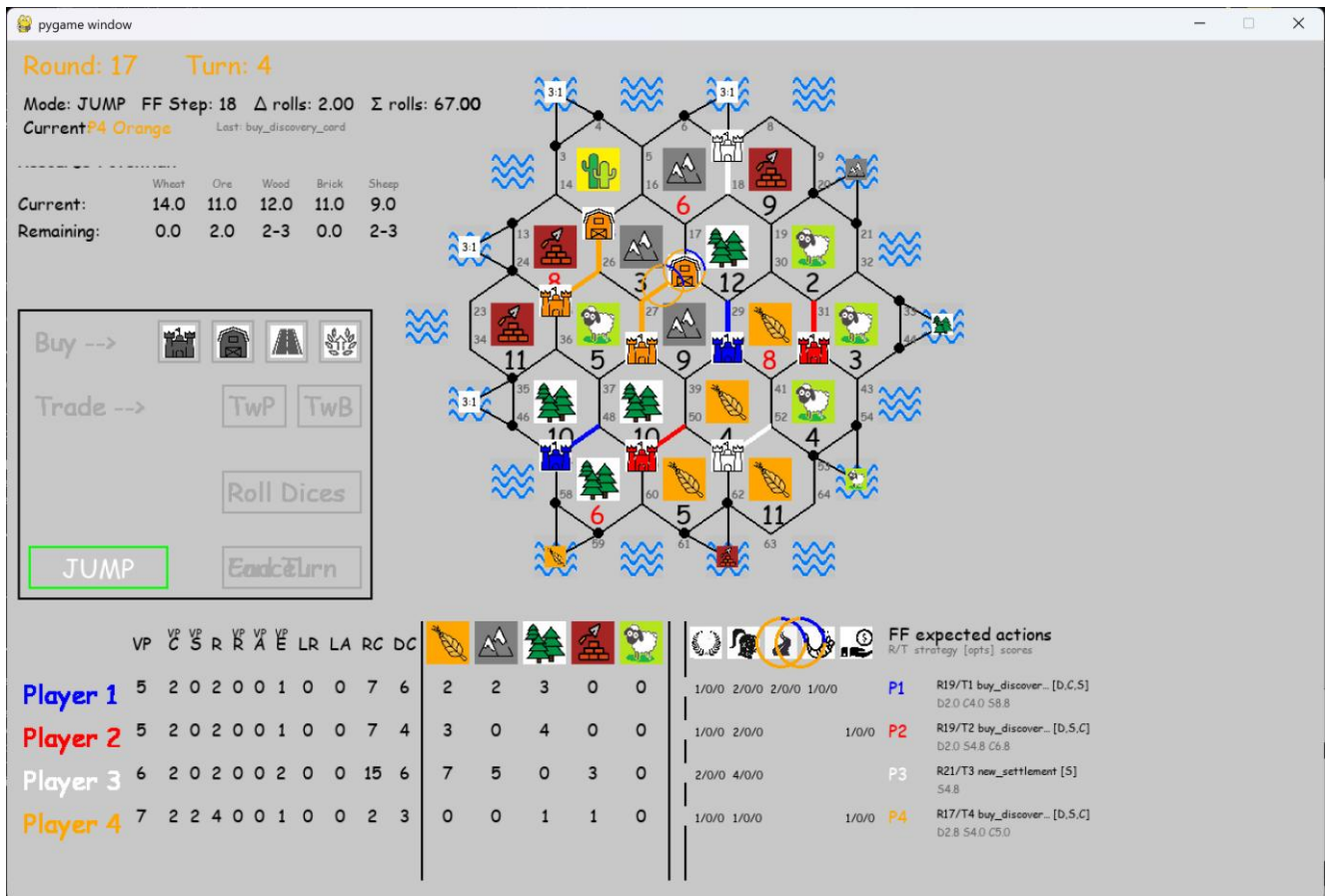
After the Initial Placement phase, v016 is not a full manual-play version. The human player no longer chooses freely between all possible Execution-phase actions. Instead, the user advances the engine with PLAY or JUMP and observes how the Expected-Hand logic converts the initial placement into future actions.

Human interaction scope in v016

The human player is fully interactive during Initial Placement. After that, the main interaction is to press PLAY or JUMP. The engine estimates, ranks, and executes actions according to Expected-Hand Feasibility Estimation and exact legality checks. This makes v016 an Initial Placement and AI-strategy evaluation version: the user watches how well the starting choices perform.

1. Roll dice or enter Action Selection for the active player through PLAY/JUMP.
2. Resolve resource production, or robber flow if a 7 is rolled.
3. Scan legal engine actions: trade, build road, build settlement, upgrade city, buy development card, or end turn.
4. Estimate timing and strategy value for relevant candidates using Expected-Hand logic.
5. Select or display the recommended action, such as BEST NOW, a wait action, or a prioritized build/buy action.
6. If the engine-selected action needs a board target, highlight the target and execute only after the normal confirmation/guard flow.

7. Execute the action only through the exact game-state guard.
8. Refresh the state and let the engine decide again, or end the turn.



8. Expected-Hand Feasibility Estimation

Expected-Hand Feasibility Estimation is the central v016 timing idea. Instead of building a full probability distribution over possible future hands, it estimates the average future hand after a number of own turns and checks whether the target cost can be paid directly or through legal bank/port trades.

Core idea

current hand + expected future production + possible trades - action cost = payable or not payable

The key module is core/resource_time_estimator.py. It uses the project resource order [Wheat, Ore, Wood, Brick, Wool]. In the GUI and write-ups, Wool may also be described as Sheep, matching the normal Catan terminology.

Component	Meaning
current_hand	The player's real resource cards at the start of the estimate.
production_pips	The total production strength per resource, based on settlements, cities, and dice numbers.
turn horizon	A future number of own turns, searched in small steps such as 0.25 turns.
expected_hand	The estimated average hand at that future horizon.
need / cost	The resource vector required for road, settlement, city, development card, or combined action.
trade_rates	The player's bank/port trade rates, usually starting at [4, 4, 4, 4, 4].

Component	Meaning
confidence	A probability-aware estimate of how reliable the affordability prediction is.

A simplified expression is:

$\text{expected_added_cards}_i(t) = \text{production_pips}_i \times \text{own_turns} \times \text{number_of_players} / 36$

$\text{expected_hand}_i(t) = \text{current_hand}_i + \text{expected_added_cards}_i(t)$

$\text{first_payable_turn} = \text{earliest } t \text{ where } \text{expected_hand}(t) \text{ can pay the cost, directly or after trades}$

Why this is useful

Expected-Hand timing is fast, explainable, and easy to debug. Every estimate can be broken down into current hand, production, required cost, trade plan, and confidence. That makes it practical for repeated JUMP/PLAY predictions and larger simulations.

9. Markov status in v016

v016 still contains substantial Markov code. This can be confusing when reading the repository, because the amount of code does not mean Markov is the dominant active v016 timing path. The reason is historical: v014 and v015 were internal project milestones. They are not separate public GitHub releases, but their important components were carried forward into the public v016 repository. v014 built the lightweight Markov direction, v015 experimented with Expected-Hand beside Markov, and v016 moved the practical timing direction further toward Expected-Hand.

Version	Main technical direction
v014 (internal)	Internal working version with improved Markov algorithm: lightweight Adaptive Expanded Evaluator direction.
v015 (internal)	Internal hybrid experiment: Expected-Hand feasibility introduced beside Markov and used for comparison/validation.
v016 (public GitHub)	Expected-Hand feasibility dominates active timing; Markov remains as retained infrastructure and historical comparison. Components developed during v014/v015 are carried forward into this repository.

Important reading note

Do not read v016 as a Markov-led version just because Markov modules are still present. For v016, the clearer description is: Expected-Hand driven timing, with Markov code retained for comparison, compatibility, and prior development history. Also, v014 and v015 are internal milestone labels, not public GitHub releases.

10. Strategy and victory-path evaluation

The project does not only estimate when an individual action becomes affordable. It also connects actions to strategic requirements and longer-term victory paths. The repository includes a 142-way resource-requirements file, which supports ranking possible routes to victory by resource needs and expected timing.

Question	Example
Can I afford this action soon?	How many own turns until I can upgrade this settlement to a city?

Question	Example
Does this action improve my future?	Will a city here increase Wheat/Ore enough to speed up a development-card path?
Which strategy is realistic for this player?	Is Largest Army viable without good Wheat/Ore/Wool access?
What is scarce or risky?	Is Brick scarce on the table, and does one player have monopoly risk?
What is the best continuation?	After building this road or settlement, which remaining strategy becomes fastest?

This layer gives BEST NOW-style recommendations more meaning. A legal action is not automatically a good action. A strategically useful action should either improve production, unlock a target, reduce expected turns to a victory path, or reduce risk.

11. Fast-forward, JUMP, PLAY, and BEST NOW

The fast-forward / recommendation loop can be understood as a prediction-and-confirmation system. JUMP or the fast-forward table estimates future events. BEST NOW or strategy matching identifies a preferred action. PLAY then checks the real state before execution. This keeps the AI explainable and prevents optimistic predictions from becoming illegal actions.

Step	Responsibility
JUMP / Fast-forward prediction	Use Expected-Hand timing to estimate future candidate actions and waiting times.
Strategy ranking	Compare candidate actions against strategy requirements and continuation value.
BEST NOW / priority action	Select the most useful available action or wait recommendation.
Exact PLAY guard	Check actual hand, bank/port trades, board legality, targets, and current phase.
Same-turn continuation	After a successful action, rescan and recompute before taking another action.

Why exact PLAY still matters

Expected-Hand estimates are averages. A player may be expected to have one Brick after several turns but still have zero Brick in the actual game path. PLAY is therefore the truth source for execution.

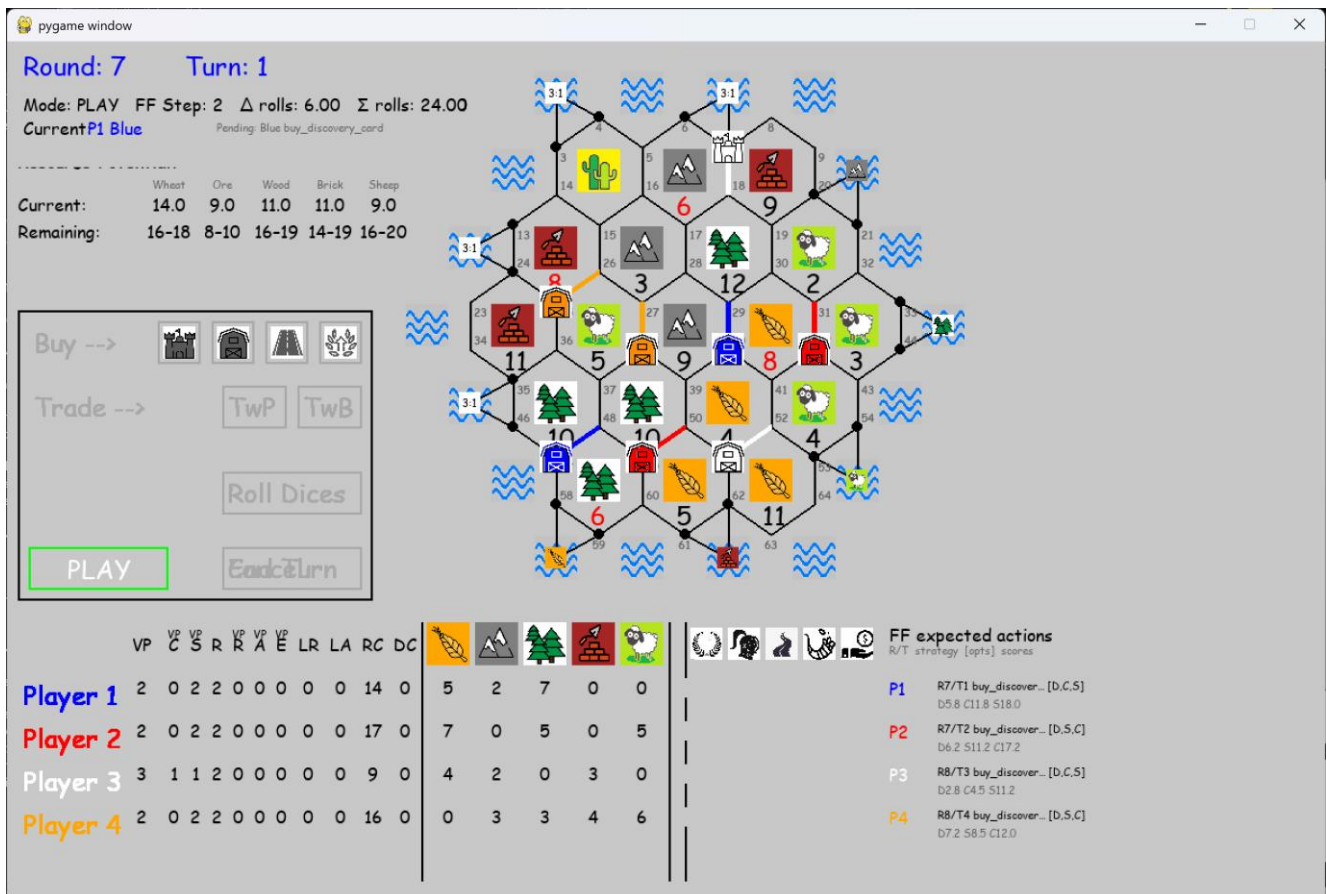
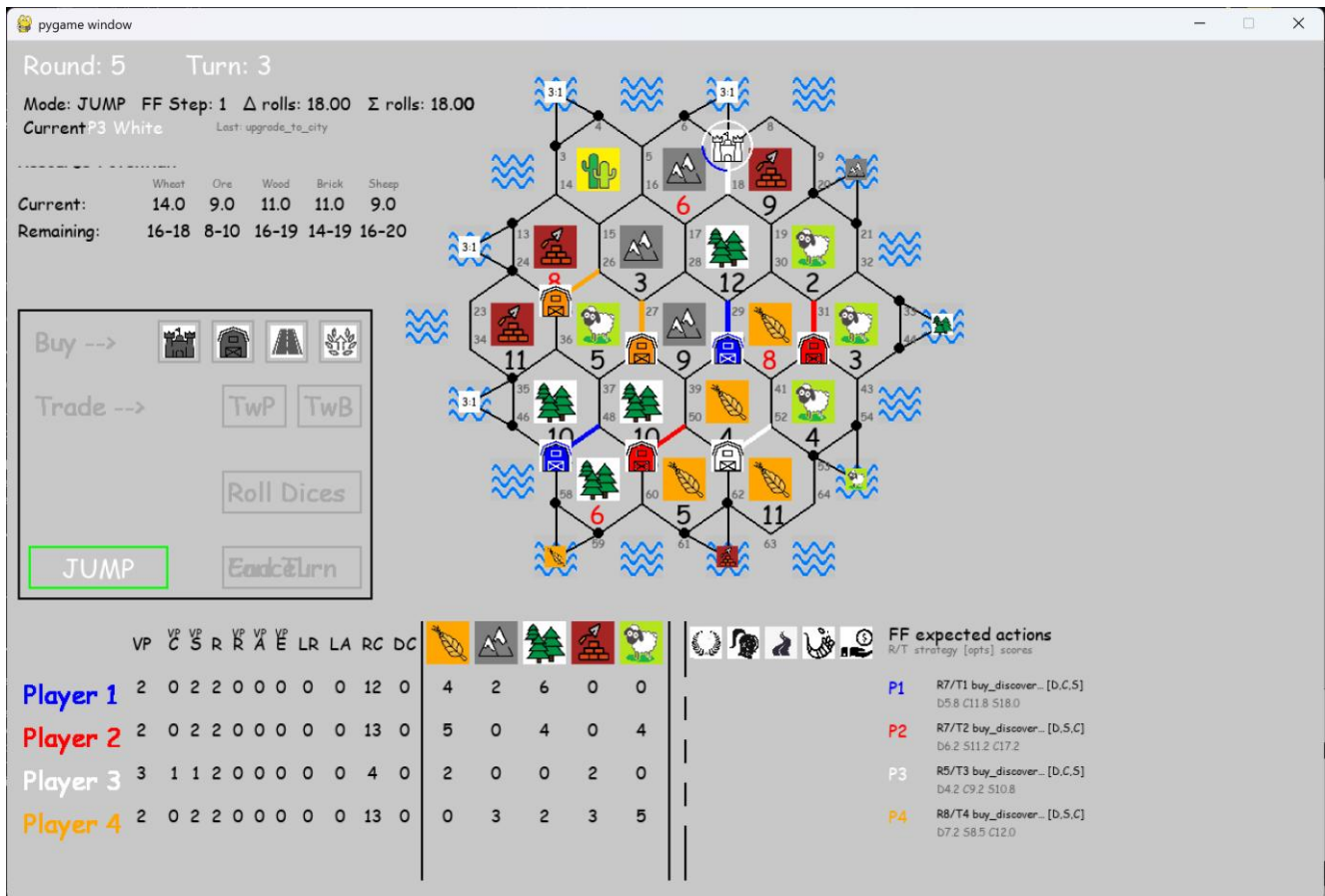


Figure 4. The top figure shows the state at R5/T3 and shows in FF expected actions (FF = Fast Forward) that the earliest Round/Turn with an action is R7/T1. Pressing the JUMP button fast-forwards to R7/T1, as shown in the bottom figure.

12. Trade, ports, and exact execution guards

Trading is important because resource timing is not only about what a player produces directly. A player with a 3:1 port or a matching 2:1 resource port may convert surplus into missing resources and complete an action earlier. Expected-Hand timing includes trade rates when estimating affordability, but exact execution still requires whole legal trades and actual cards.

Trade concept	Meaning in v016
Bank trade	Default 4:1 conversion from surplus cards to a missing card.
3:1 port	Improves generic trade rate where the player owns the port.
2:1 resource port	Improves the matching resource only, e.g., strong Wheat plus 2:1 Wheat port.
Continuous timing estimate	Useful for future average estimates, but not the same as a real integer trade.
Exact current-hand guard	At PLAY time, the game must use actual whole trades and real resource counts.

The distinction between estimate and execution is one of the most important design safeguards in v016. Expected-Hand may predict that a road, city, settlement, or development card is likely feasible at a future point. PLAY determines whether the game can actually perform it now.

13. Debugging, reports, and explainability

A major strength of the v016 direction is explainability. Expected-Hand timing can show why an action is estimated at a certain turn count. Strategy and victory-path evaluation can show why a route is viable or not viable. The GUI and reports can display enough information to debug wrong recommendations without reverse-engineering a black box.

Output / diagnostic	Useful for
expected_hand	Checking whether the estimated future hand covers the cost.
need_vector	Showing which resources the target action or strategy still requires.
trade plan	Showing how surplus resources might be converted into missing resources.
confidence label	Distinguishing high-confidence direct production from low-confidence scarce/trade-dependent estimates.
scarcity table	Showing table_scarce, scarce_for_players, players_without_access, and monopoly_risk.
victory-path / action-evaluation output	Ranking candidate actions and longer-term paths by Expected-Hand timing, strategic resource needs, and continuation value.
fast-forward table	Displaying predicted action timing, source, target, and execution status.

Design principle

Every recommendation should be explainable as a chain: player state -> resource timing -> strategy requirement -> legal action -> exact execution check.

14. Known limitations and next steps

- Expected-Hand uses averages. It is faster and easier to debug than full probability-state modeling, but it does not represent the full distribution of possible hands.
- Confidence scoring and safety margins remain important, especially for scarce resources and multi-resource costs.
- Full manual human Execution-phase decision-making is not implemented in v016. After Initial Placement, the human player mainly uses PLAY/JUMP to advance the Expected-Hand engine and observe how well the initial placement performs.
- The presence of Markov code can make the repository look more Markov-led than v016 actually is. Documentation should state this clearly.
- Trade with Bank and port logic are central to timing, but player-to-player trade logic should remain clearly marked as experimental unless fully integrated in the GUI flow.
- Strategy and victory-path evaluation should continue to exclude unrealistic Largest Army or Longest Road paths when a player lacks the required resource access.
- More automated validation is useful: compare predicted timing, exact PLAY success, actual action chosen, and longer-term strategic outcome.

Suggested next research question

Can probability-aware Expected-Hand feasibility replace light Markov as the primary engine for fast Catan simulation while maintaining acceptable prediction quality?

15. Acknowledgements and external inspiration

This project was developed as an educational AI/game-engine project. The Markov-chain direction was inspired by Lauren Nagel's work on analyzing The Settlers of Catan using transition matrices. The victory-path component was inspired by PlayerOne's Board Game Analysis series on the 102/143 ways to win at Catan. These works influenced the project's thinking about probability, timing, and strategic paths; they are not copied code.

OpenAI's ChatGPT was used as a writing and coding assistant during development discussions. The author remains responsible for final code decisions, testing, interpretation, and publication.

Annex A. Module overview

Module / file	Short description
main.py	Starts the Pygame application and initializes the game loop.
core/board.py	Defines tiles, intersections, roads, harbors, connectivity, and board geometry.
core/player.py	Stores resources, pieces, settlements, cities, roads, development cards, trade rates, and player statistics.
core/game.py	Central game orchestrator for phases, turns, dice rolls, player state, board state, saves/loads, and execution flow.
core/constants.py	Stores resource order, build costs, configuration flags, image/sound identifiers, and helper constants.
core/initial_placement.py	Coordinates Initial Placement and chooses which placement algorithm to use.
core/algorithms_initial_placement.py	Implements pips, ports, five-strategy, and other placement algorithms.
core/markov_evaluator.py	Retained Markov-style evaluator infrastructure from earlier versions and comparison paths.
core/resource_time_estimator.py	Expected-Hand engine for estimating first payable turn, payability after trades, confidence, and resource-timing explanations.
core/victory_path_evaluator.py	Evaluates strategic victory paths using the 142-way resource-requirements data and supports longer-term strategy ranking.
core/action_evaluator.py	Evaluates possible actions and connects Expected-Hand feasibility to strategy recommendations.
core/fast_forward.py	Supports fast-forward/JUMP timing tables, predicted action scheduling, and recommendation refresh behavior.
core/player_outlook.py	Analyzes possible future settlements, roads, and contested targets.
catan_142_ways_resource_requirements.csv	Structured resource requirements for strategic victory-path analysis.
PlayBoard 08_Apr_2026_13_33_06.txt	Saved playboard used to initialize a known board setup.
gui/gui.py	Main renderer for board, scoreboard, panels, and drawing helpers.
gui/event_handler.py	Routes mouse clicks between buttons, board guidance, panels, and confirmations.
gui/gui_human_player.py	Renders and updates human action buttons.
gui/gui_constants.py	Centralizes GUI positions, fonts, colours, image references, and sound references.

Annex B. Key classes and responsibilities

This annex is intentionally written at a high level. It can be expanded after another code pass if a complete class-by-class inventory is required.

Class / object type	Likely responsibility
Game	Owns the overall game state, phase, players, board, turn details, and execution-phase transitions.
Board	Owns the tile, intersection, road, and port structures and provides board geometry.
Tile	Represents one hex tile, including terrain/resource type and dice value.
Intersection	Represents one vertex, including occupancy, buildability, adjacent tiles, and possible port access.
Road	Represents one edge between intersections and tracks ownership/build status.
Player	Represents a player's hand, pieces, roads, settlements, cities, ports, development cards, and score-related state.
TurnDetails / dashboard objects	Track per-turn resource changes, scoreboard details, and debug state.
Expected-Hand result dataclasses	Represent payability, confidence, and action-time estimates in <code>resource_time_estimator.py</code> .
GUI	Draws the board, scoreboard, panels, pieces, highlights, and visual feedback.
GUIHumanPlayer / button objects	Manage button display and active/inactive state for human actions.
EventHandler	Routes mouse input to the correct game or GUI subsystem.